THE UNIVERSITY OF MELBOURNE

COMP90038 Algorithms and Complexity

Lecture 6: Recursion

Olya Ohrimenko
(with thanks to Harald Søndergaard)

Recursion

- We've already seen some examples
- A very natural approach when the data structure is recursive (e.g. lists, trees)
- But also examples of naturally recursive array processing algorithms
- Next week we'll express **depth first graph traversal** recursively (the natural way); later we'll meet other examples of recursion too

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example: Factorial

- n!: we can use recursion (left) or iteration (right)

```
function FAC(n)
    if n = 0 then
        return 1
    return FAC(n − 1) ∗ n
```

$$
\begin{aligned}
F(5) &= F(4) \cdot 5 \\
&= (F(3) \cdot 4) \cdot 5 \\
&= ((F(2) \cdot 3) \cdot 4) \cdot 5 \\
&= (((F(1) \cdot 2) \cdot 3) \cdot 4) \cdot 5 \\
&= ((((F(0) \cdot 1) \cdot 2) \cdot 3) \cdot 4) \cdot 5 \\
&= ((((1 \cdot 1) \cdot 2) \cdot 3) \cdot 4) \cdot 5 \\
&= 120
\end{aligned}
$$

```
function FAC(n)
    result ← 1
    while n > 0 do
        result ← result ∗ n
        n ← n − 1
    return result
```

n: 0

result: 120

Iterative version normally preferred since it is constant space

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example: Fibonacci Numbers

- To generate the *n*th number of sequence: 1 1 2 3 5 8 13 21 34 55 ...

```
function FIB(n)
    if n = 0 then
        return 1
    if n = 1 then
        return 1
    return FIB(n − 1) + FIB(n − 2)
```


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example: Fibonacci Numbers



THE UNIVERSITY OF MELBOURNE

- To generate the *n*th number of sequence: 1 1 2 3 5 8 13 21 34 55 …

function FIB(n)

…

return FIB($n-1$) + FIB($n-2$)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example: Fibonacci Numbers

- To generate the *n*th number of sequence: 1 1 2 3 5 8 13 21 34 55 …

```
function FIB(n)
    if n = 0 then
        return 1
    if n = 1 then
        return 1
    return FIB(n − 1) + FIB(n − 2)
```

Follows the mathematical definition of Fibonacci numbers very closely.

Easy to understand

But performs lots of redundant computation

Basic operation: addition

Complexity is **exponential** in *n*

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Fibonacci Again

- Of course we only need to remember the latest two items. Recursive version: left; iterative version: right

```
function FIB(n, a, b)
    if n = 0 then
        return a
    return FIB(n − 1, a + b, a)
```

```
function FIB(n)
    a ← 1
    b ← 0
    while n > 0 do
        t ← a
        a ← a + b
        b ← t
        n ← n − 1
    return a
```

Initial call: Fib(***n***, 1, 0)

Time complexity of both solutions is linear in ***n***

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Tracing Recursive Fibonacci



THE UNIVERSITY OF MELBOURNE

```
function FIB(n, a, b)
    if n = 0 then
        return a
    return FIB(n − 1, a + b, a)
```

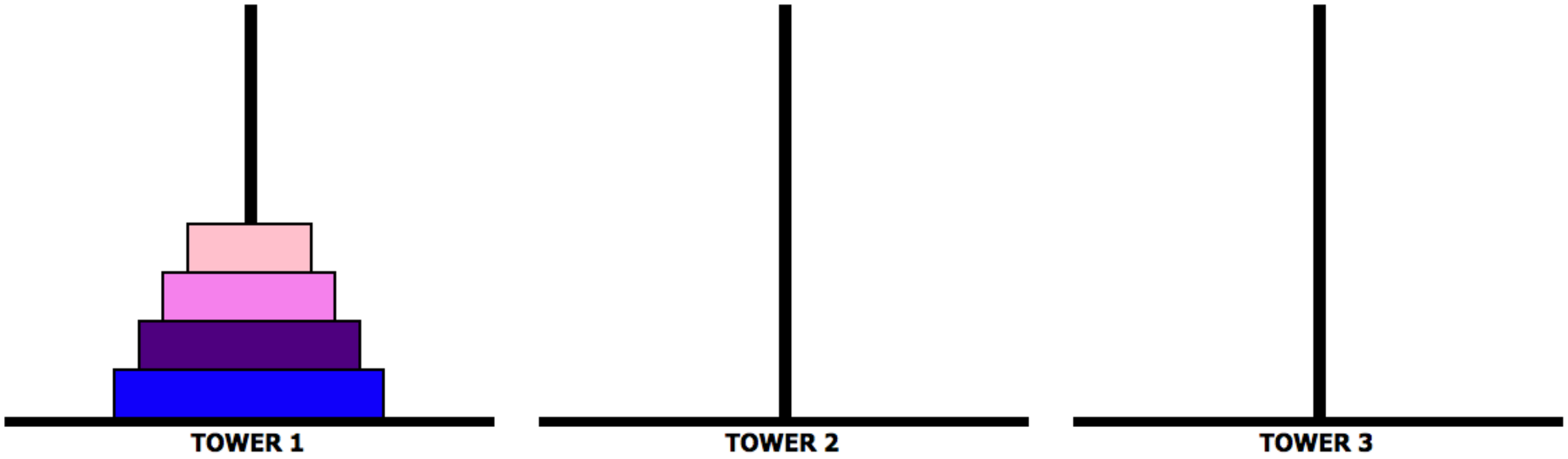
Initial call: Fib(***5***, 1, 0)

Fib(5,1,0)
= Fib(4,1,1)
= Fib(3,2,1)
= Fib(2,3,2)
= Fib(1,5,3)
= Fib(0,8,5) = 8

Fibonacci Sequence:
1 1 2 3 5 8 …

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Tower of Hanoi

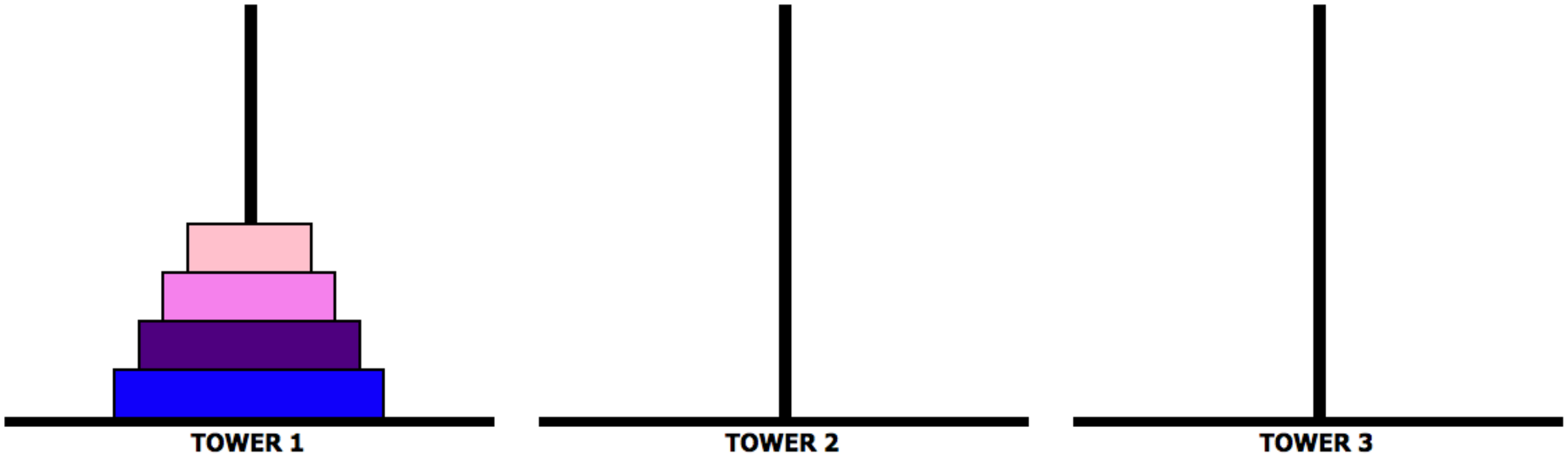


Init Aux Fin

Move *n* disks from *Init* to *Fin*. A larger disk can never be placed on top of a smaller one.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Tower of Hanoi: Recursive Solution

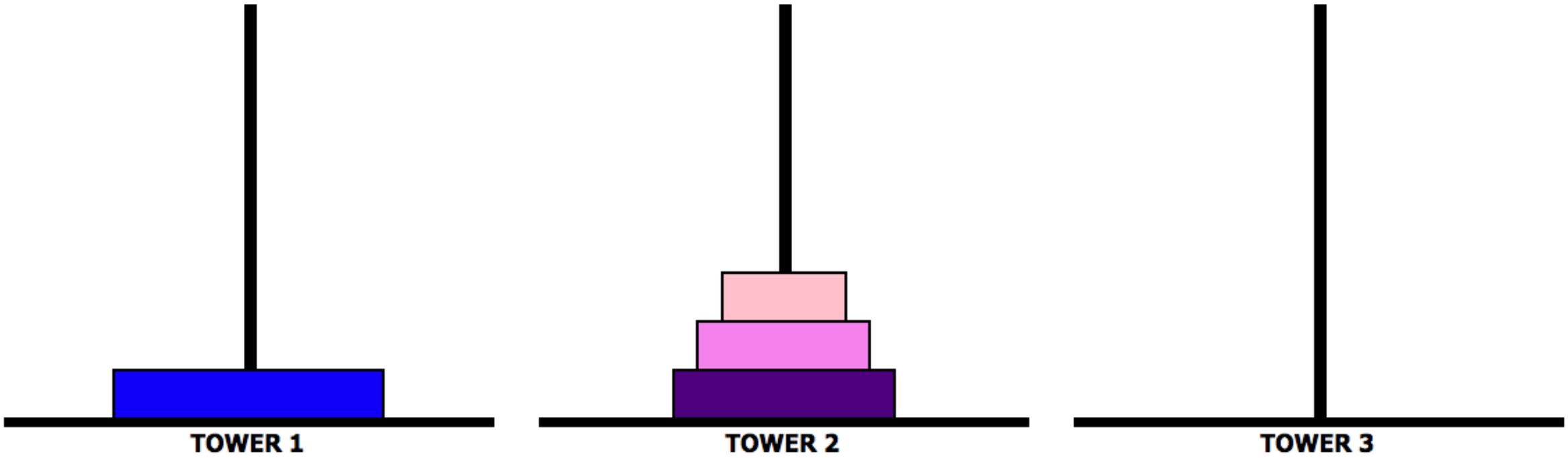


Init Aux Fin

Move ***n-1*** disks from Init to Aux.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Tower of Hanoi: Recursive Solution

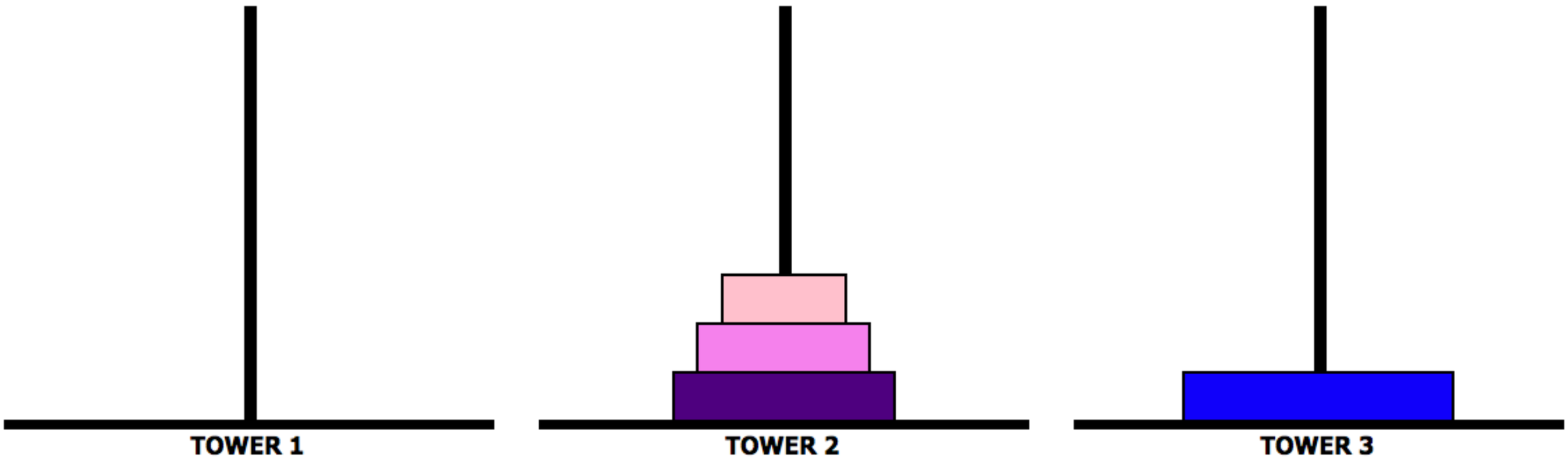


Init Aux Fin

Move *n-1* disks from Init to Aux.
Then move the *n*th disk to Fin.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Tower of Hanoi: Recursive Solution

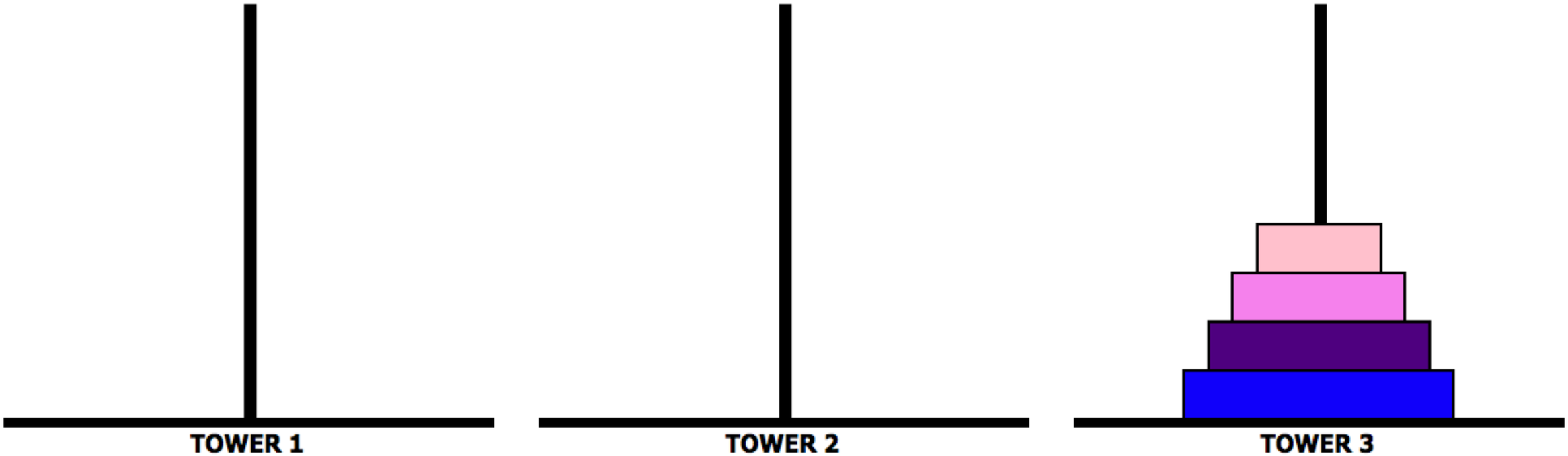


Init Aux Fin

Move ***n-1*** disks from Init to Aux.
Then move the ***n***th disk to Fin.
Then move the ***n-1*** disks from Aux to Fin.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Tower of Hanoi: Recursive Solution



Init Aux Fin

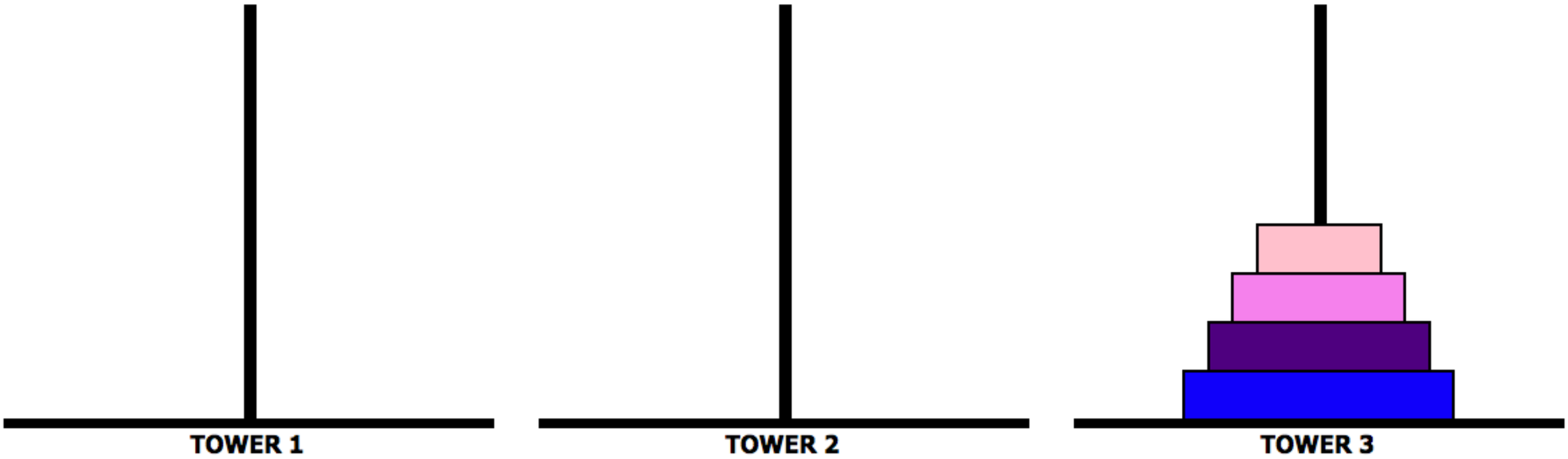
Move ***n-1*** disks from Init to Aux.
Then move the ***n***th disk to Fin.
Then move the ***n-1*** disks from Aux to Fin.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Tower Of Hanoi: Recursive Algorithm

THE UNIVERSITY OF MELBOURNE

function HANOI(n, *init*, *aux*, *fin*)
  if $n > 0$ **then**
    HANOI($n - 1$, *init*, *fin*, *aux*)
    Move one disk from *init* to *fin*
    HANOI($n - 1$, *aux*, *init*, *fin*)



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Tracing Tower of Hanoi Recursive Algorithm

THE UNIVERSITY OF MELBOURNE

https://www.cs.cmu.edu/~cburch/survey/recurse/hanoiex.html

Assume n = 64 disks
Moving a disk at a rate of one per second, using the smallest number of moves, would take $2^{64} - 1$ seconds or roughly 585 billion years to finish.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

A Challenge:
Coin Change Problem

THE UNIVERSITY OF MELBOURNE

- There are 6 different kinds of Australian coin
- In cents, their values are: 5, 10, 20, 50, 100, 200
- In how many different ways can I produce a handful of coins adding up to $3?
- For example, 3 different ways would be: {100,100,100}, {200,50,50}, {50,50,50,50,50,50}.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

A Challenge:
Coin Change Problem

THE UNIVERSITY OF MELBOURNE

- There are 6 different kinds of Australian coin
- In cents, their values are: 5, 10, 20, 50, 100, 200
- In how many different ways can I produce a handful of coins adding up to $3?
- This is not an easy problem!
- Key to solving it is to find a way to break it down into simpler sub-problems

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Coin Change Problem: Decomposition

$3





Use $2 coin

No

$3

made from



Yes



+ $1

made from



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Coin Change Problem: Decomposition

The number of ways of making $3 is therefore:

the number of ways of making $3 without using a $2 coin

+

the number of ways of making $1

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Coin Change Problem: Partial Algorithm



THE UNIVERSITY OF MELBOURNE

```
function WAYS(amount, denominations)
    // … base cases ….
    d ← selectLargest(denominations)
    return  WAYS(amount, denominations \ {d}) +
            WAYS(amount − d, denominations)
```

For example:

Ways(300, {5,10,20,50,100,200}) =
  Ways(300, {5,10,20,50,100}) +
  Ways(100, {5,10,20,50,100,200})

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Coin Change Problem: Base Cases



- Each time we recurse, we decrease either:
  - amount (by subtracting some quantity from it), or
  - demonisations (by removing an item from the set)

function WAYS(amount, denominations)

- Consider each of these separately.
  - amount = 0: there is **one** way (using no coins)
  - amount < 0: there are **no ways** to make this
  - denominations = ∅ (and amount > 0): there are **no ways** to make this amount

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Coin Change Problem: Full Recursive Algorithm



THE UNIVERSITY OF MELBOURNE

```
function WAYS(amount, denominations)
    if amount = 0 then
        return 1
    if amount < 0 then
        return 0
    if denominations = ∅ then
        return 0
    d ← selectLargest(denominations)
    return WAYS(amount, denominations \ {d}) +
                WAYS(amount − d, denominations)
```

Initial call: WAYS(amount, {5, 10, 20, 50, 100, 200}).

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Solution and its Complexity

THE UNIVERSITY OF MELBOURNE

- Although our recursive algorithm is short and elegant, it is not the most efficient way of solving the problem.

- Its running time grows **exponentially** as you grow the input amount.

- More efficient solutions can be developed using **memoing** or **dynamic programming**—more about that later (around Week 10).

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Next Time…

- Graphs, trees, graph traversal and allied algorithms.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License